TIP Another option is to make a tiny change in the OpenAPI document so that NSwag updates the generated code when you build your project. Then you can revert the OpenAPI document change.

After you've persuaded NSwag to regenerate the client, you should update your code to use the new features. You can remove the Newtonsoft.Json reference from your csproj file and update your Program.cs as shown in the following listing.

Listing 11.8 Using the updated NSwag client

```
using Fruit;

IFruitClient client = new FruitClient(          ❶
    new HttpClient() { BaseAddress =            ❷
        new Uri("https://localhost:7186") });   ❷

Fruit.Fruit created = await client.FruitPOSTAsync("123",
    new Fruit.Fruit { Name = "Banana", Stock = 100 });
Console.WriteLine($"Created {created.Name}");

Fruit.Fruit fetched = await client.FruitGETAsync("123");
Console.WriteLine($"Fetched {fetched.Name}");
```

❶ FruitClient now implements IFruitClient.

❷ Sets the base address on HttpClient instead of passing as a constructor argument

If you updated the operation IDs for your API endpoints using `WithName()`, you may be a little surprised to see that you still have the ugly `FruitPOSTAsync` and `FruitGETAsync` methods, even though you regenerated the client. That's because the OpenAPI description saved to your project is downloaded only once, when you initially add it. Let's look at how to update the local OpenAPI document to reflect the changes to your remote API.

11.4.5 Refreshing the OpenAPI description

In this section you'll learn how to update the OpenAPI description document saved to your project that's used for generation. This document doesn't update